Solution: Removing Stars From a String

Problem Statement

Given a string `s`, where `*` represents a star. We can remove a star along with its closest non-star character to its left in a single operation.

The task is to perform as many such operations as possible until all stars have been removed and return the resultant string.

Examples

Example 1

- Input: `"abc*de*f"`
- Expected Output: `"abdf"`
- Description: We remove `c` along with `*` to get `"abde*f"`, then remove `e` along with `*` to get `"abdf"`

Example 2

- Input: `"a*b*c*d"`
- Expected Output: `"d"`
- Description: We remove `a` along with `*` to get `"b*c*d"`, then remove `b` with `*` to get `"c*d"`, then remove `c` with `*` to get `"d"`.

Example 3

- Input: `"abcd"`
- Expected Output: `"abcd"`
- Description: As there is no `*`, the string remains the same.

Solution

To solve this problem, we can use a stack to efficiently handle the removal of characters caused by asterisks. As we iterate through the string, we push each non-asterisk character onto the stack. When we encounter an asterisk, we pop the top character from the stack, effectively removing the last non-asterisk character encountered. This process mimics the removal of characters as specified by the asterisks.

After processing the entire string, the remaining elements in the stack represent the characters that survived the removal process. Finally, we reverse the stack to reconstruct the string in the correct order, as the stack will have the characters in reverse order of their original appearance.

Detailed Step-by-Step Algorithm

1. Initialize an empty stack
2. Iterate over each character in the input string
3. For each character, check if it's a star and the stack is not empty
   - If it is, pop the top character from the stack
   - If it's not a star, push the character into the stack
4. After the iteration, convert the stack into a string. The order of the characters in the stack is the reverse of the final string, so reverse the stack's string before returning
5. Return the final string

Algorithm walkthrough

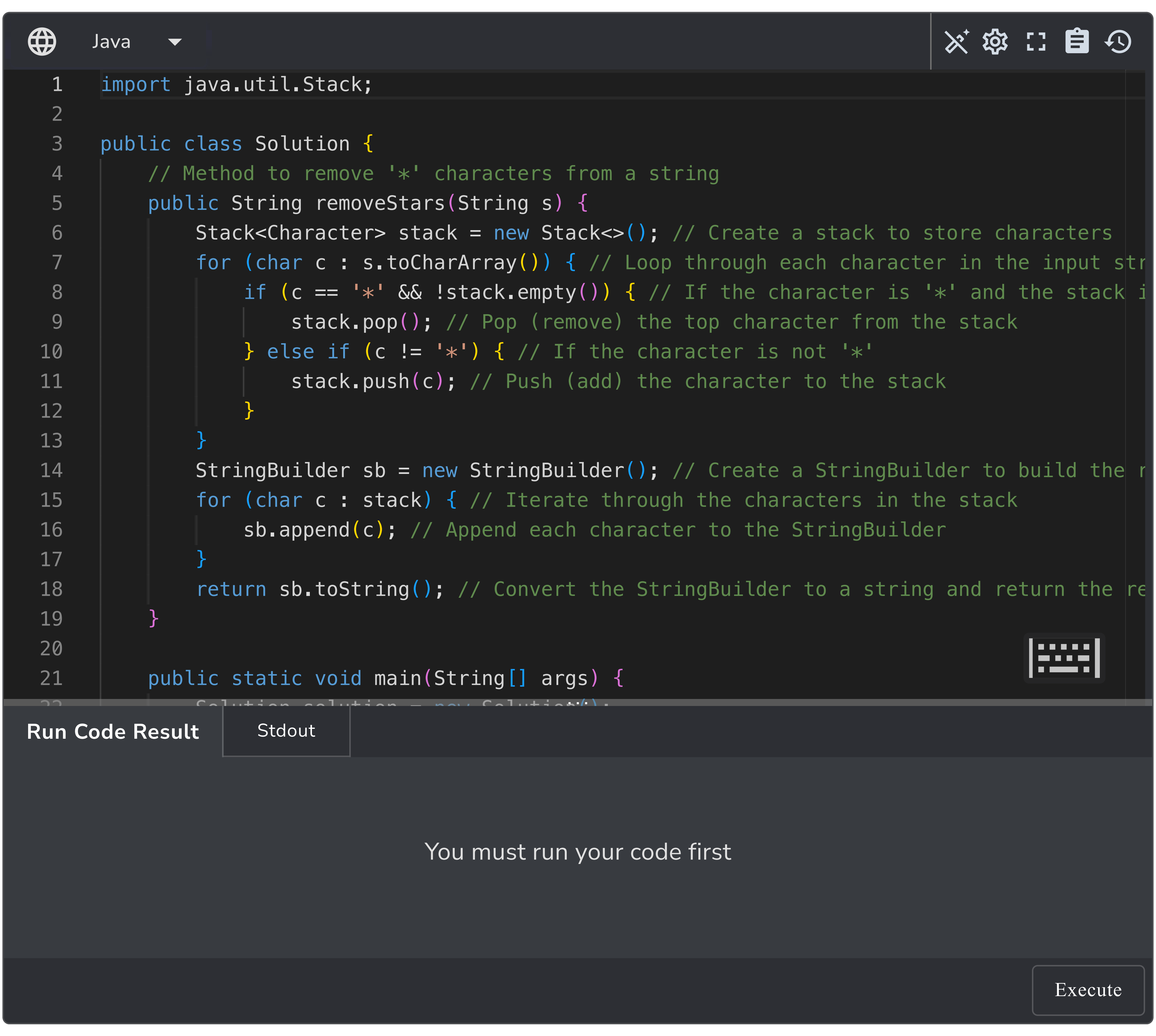
Let's consider the input `abc*de*f`, and observe how algorithm works.

1. **Initialize an Empty Stack:** We start with an empty stack to store characters.
2. **Iterate Through the String:** We process each character in the input string "abc*de*f" one by one.
   - **a:** Since 'a' is not an asterisk, we push it onto the stack. Stack: [a]
   - **b:** Push 'b'. Stack: [a, b]
   - **c:** Push 'c'. Stack: [a, b, c]
   - ***:** Encountering an asterisk, we pop the top character ('c') from the stack. Stack: [a, b]
   - **d:** Push 'd'. Stack: [a, b, d]
   - **e:** Push 'e'. Stack: [a, b, d, e]
   - ***:** Another asterisk, pop the top character ('e'). Stack: [a, b, d]
   - **f:** Push 'f'. Stack: [a, b, d, f]
3. **Reconstruct the Resulting String:** After processing all characters, the stack contains [a, b, d, f]. We need to convert this stack into a string. Since the stack is in reverse order, we build the string from the bottom of the stack upwards, resulting in "abdf".

The final output for the input "abc*de*f" using this algorithm is "abdf".

Code

Here is how we can implement this algorithm:



```java
import java.util.Stack;

public class Solution {
    // Method to remove '*' characters from a string
    public String removeStars(String s) {
        Stack<Character> stack = new Stack<>(); // Create a stack to store characters
        for (char c : s.toCharArray()) { // Loop through each character in the input str
            if (c == '*' && !stack.empty()) { // If the character is '*' and the stack i
                stack.pop(); // Pop (remove) the top character from the stack
            } else if (c != '*') { // If the character is not '*'
                stack.push(c); // Push (add) the character to the stack
            }
        }
        StringBuilder sb = new StringBuilder(); // Create a StringBuilder to build the r
        for (char c : stack) { // Iterate through the characters in the stack
            sb.append(c); // Append each character to the StringBuilder
        }
        return sb.toString(); // Convert the StringBuilder to a string and return the re
    }

    public static void main(String[] args) {
```

Time and Space Complexity

The time complexity of the algorithm is $O(n)$, where n is the length of the string. This is because we perform a single pass over the string.

The space complexity of the algorithm is $O(n)$, where n is the length of the string. In the worst case, if there are no stars, the stack will contain all the characters of the string.